

Security and Usability Trade-offs Online Banking App: Monobank

WM241 Human Behaviour in Cyber Systems

Author Student ID: u5602780

Module Tutor: Dr. Elzbieta Titis

December 3, 2025

Written Report

Word Count: 1021

Contents

1	Introduction	2
2	Model Usage Scenario	3
2.1	Scenario Description	3
3	Model Threat Scenario	4
3.1	Scenario Description and motivators	4
3.2	Summary of Findings	6
4	Recommendations	6
4.1	Risk-Level Matrix	6
4.2	Actionable Recommendations	6
5	Conclusion	7
A	Backend Code	8

1 Introduction

Monobank is a widely used online banking app in Ukraine, with around ten million users, approximately a quarter of the country's population [4]. Its creators have built a system somewhat similar to Monzo, a well-known online bank in the UK.

Balancing security and usability is extremely important. As is well known, users, like water, tend to follow the path of least resistance. It is therefore the developer's responsibility to design systems in such a way that user convenience or laziness does not compromise security.

In this report, I will examine the usability and security risks of the money transfer and payee addition features, referencing [3].

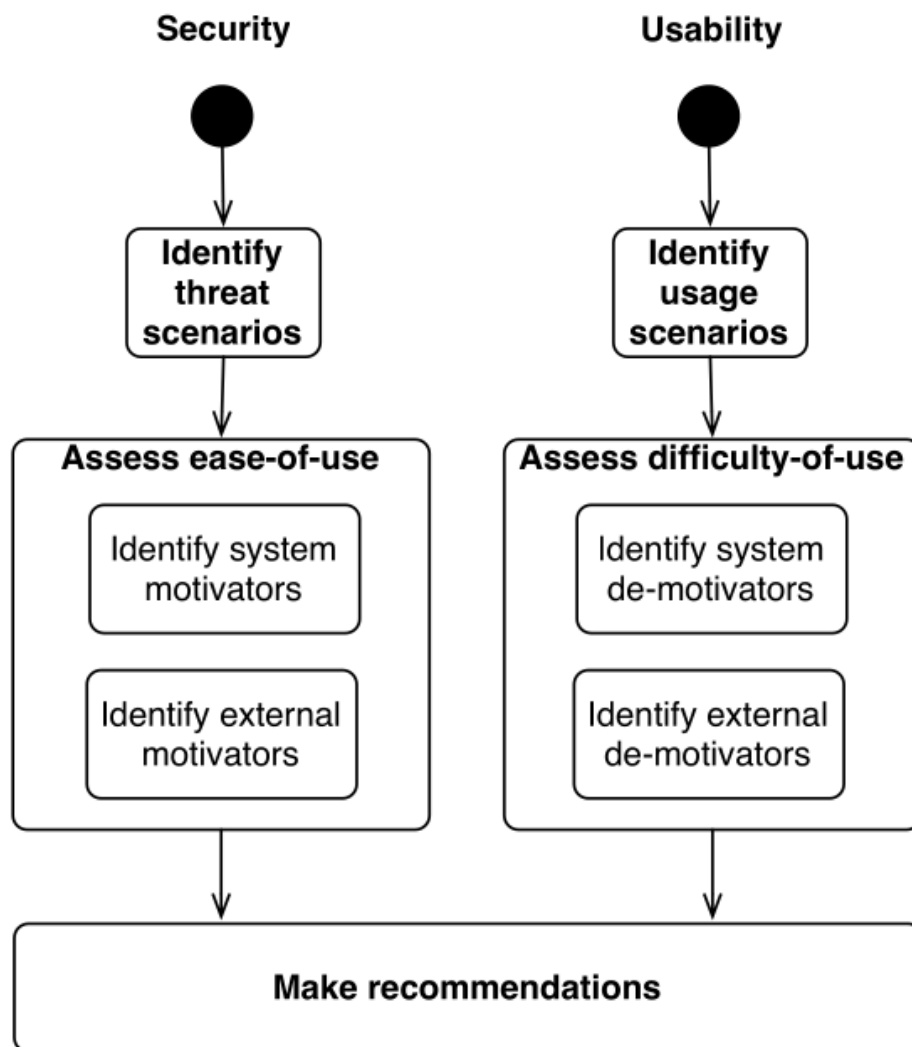


Figure 1: Process for security-usability analyses from kaunda model

2 Model Usage Scenario

2.1 Scenario Description

Sending money to contacts is an important feature of Monobank [4], and adding people to your contacts is equally significant. This functionality is especially important for Ukrainians, as there is a strong culture of donating to charities and individuals [5].

De-motivators are factors that make legitimate usage scenarios harder, discouraging proper secure behaviour.

System de-motivators:

- **SDM1:** The 16-digit card number is difficult to memorise or check visually, increasing the cognitive load during manual entry.
- **SDM2:** Lack of clear user feedback (no name, no icon, no confirmation) increases uncertainty and reduces confidence in performing the secure action correctly.
- **SDM3:** The system provides no warnings for potentially incorrect card numbers beyond the Luhn check, making detection of subtle errors difficult.

External de-motivators:

- **EDM1:** Users may feel social pressure to complete a transfer quickly, especially when donating or paying someone while in conversation.
- **EDM2:** Poor lighting conditions or screen glare make digit comparison more difficult.
- **EDM3:** Age-related or situational decreases in attention (stress, urgency, emotional context) make secure careful entry harder to maintain.

Even though credit card numbers utilise the Luhn [6] algorithm, which allows for simple typo checking, it does not cover all possible input errors. An appendix script was used to take an original card number and identify similar valid combinations. For example, given the charity card number:

4149 4990 7890 9809

the script found three other valid card numbers differing by minor typos (transpositions):

- 414949**09**78909809
- 4149499078**09**9809
- 414949907899**08**09

It is important to note that the Ukrainian banking system does not use sort codes or sharecodes. All transactions require only a 16-digit card number.

All of the above mistyped card numbers correspond to existing cards from “Private Bank.”

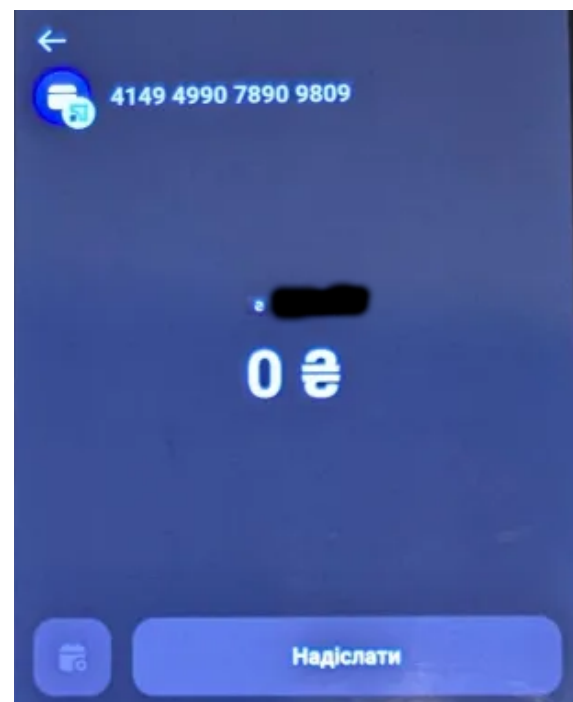
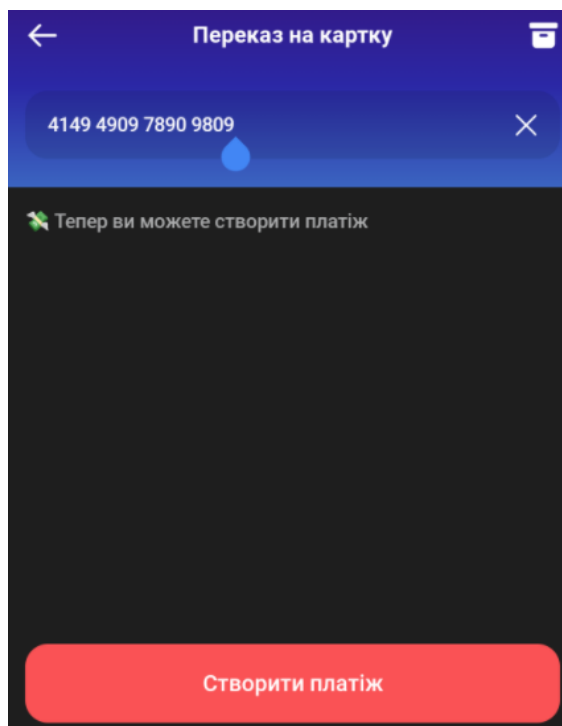
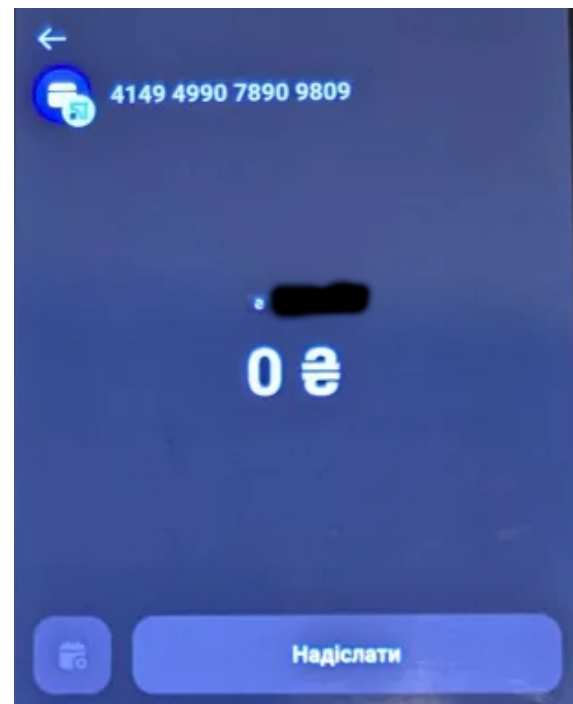
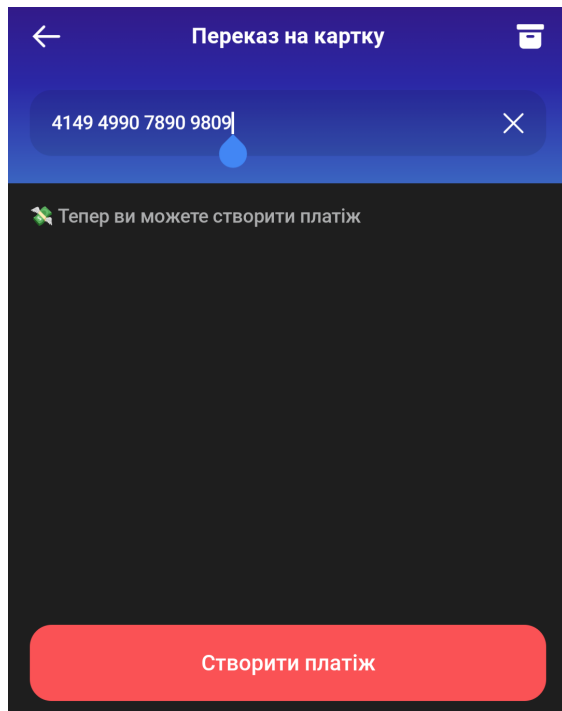


Figure 2: Original and mistyped debit card transfers

3 Model Threat Scenario

3.1 Scenario Description and motivators

motivators – factors that increase the likelihood of a user performing a threat scenario

System motivators:

- **SM1:** Manual entry of long 16-digit card numbers makes it faster and easier for users

to type quickly without double-checking, increasing the likelihood of mistyping.

- **SM2:** The Monobank app does not display the recipient’s name or photo, nor does it present a confirmation step such as “Are you sure?”, which reduces user vigilance.
- **SM3:** Because the app allows immediate confirmation, users become conditioned to proceed quickly without verifying accuracy.

External motivators:

- **EM1:** Environmental distractions such as noise, movement, or stressful surroundings encourage users to prioritise speed over accuracy.
- **EM2:** Cognitive conditions such as dyscalculia make it more difficult to accurately process and verify long numeric strings.
- **EM3:** Fatigue, alcohol consumption, or multitasking reduces attention [2] and increases reliance on habitual fast typing.

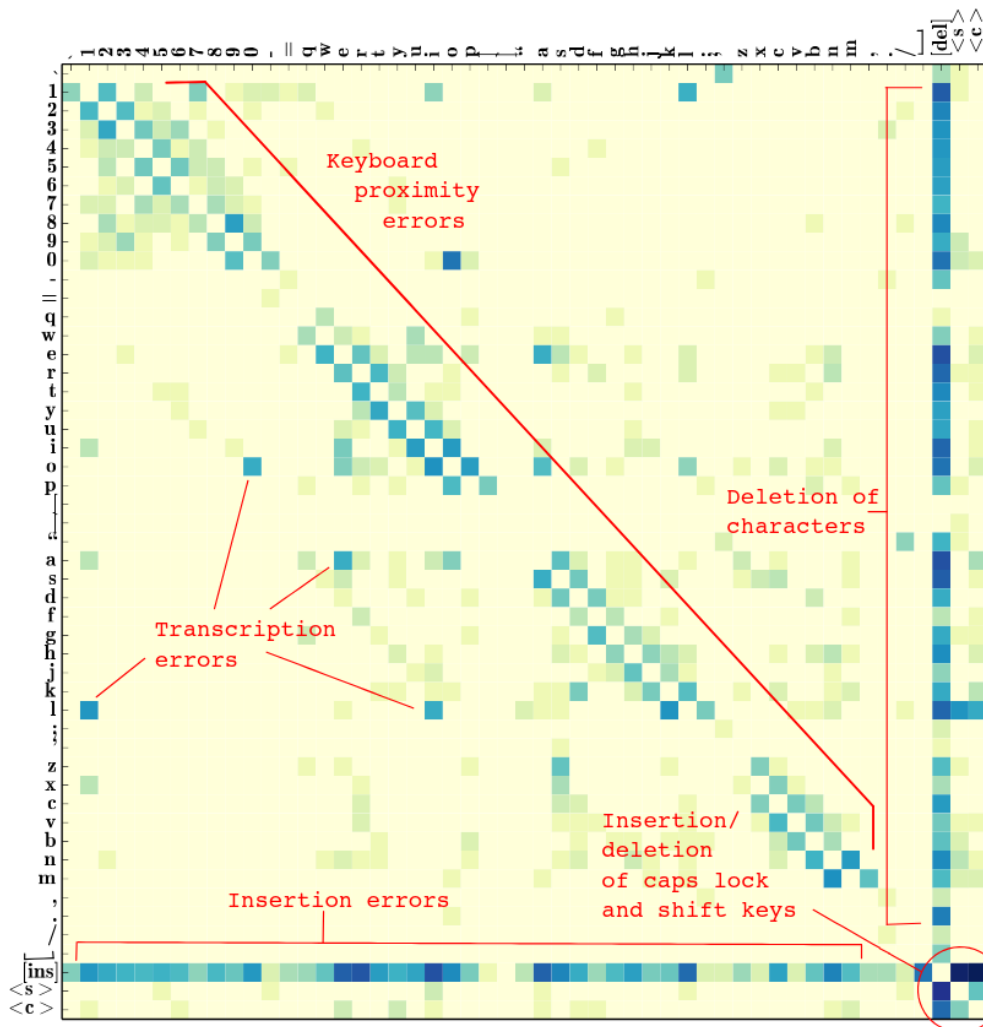


Figure 3: Figure 1 from the *Password Typos* paper illustrating character substitution frequency.

The top-left corner of this figure shows how often nearby characters, such as 0 and 9, are replaced with one another. This is particularly relevant, as the way the Luhn checksum works means that swapping these digits can still produce a valid card number. The same

paper indicates that approximately 5.5% of unique passwords were mistyped at least once [1]. Due to the lack of dedicated research on mistyped bank account numbers, this serves as the best available reference for extrapolation.

3.2 Summary of Findings

The root cause of insecure behaviour lies in the lack of user feedback and confirmation, combined with predictable human error patterns and environmental distractions.

4 Recommendations

4.1 Risk-Level Matrix

A simple risk matrix can rank motivators and demotivators by likelihood and impact. Prioritisation should focus on issues with both high likelihood and high impact, such as mistyping card numbers, since they directly lead to financial loss.

Impact\Likelihood	Low	Moderate	High
Low	N/A	EDM2	N/A
Moderate	N/A	EDM1	SDM1 EDM3 SM3 EM1
High	EM2	EM3	SDM3 SM1 SM2

Table 1: Modern Risk Level Matrix

4.2 Actionable Recommendations

Summary of High-Risk Motivators and De-Motivators The following factors were identified as the highest risk due to their combined likelihood and impact:

- **SDM1:** 16-digit card numbers are difficult to memorise or verify visually.
- **SDM3:** No warnings beyond the Luhn check allow subtle errors to go undetected.
- **EDM3:** Stress, urgency, or emotional context significantly reduce user attention.
- **SM1:** Manual entry of long card numbers encourages fast, unchecked typing.
- **SM2:** The absence of recipient name/photo reduces user vigilance.
- **SM3:** Immediate confirmation screens condition users to skip verification steps.
- **EM1:** Environmental distractions lead users to prioritise speed over accuracy.
- **EM3:** Fatigue, alcohol consumption, or multitasking reduce attention.

Human factors such as stress, fatigue, or alcohol cannot realistically be restricted by the application. Therefore, mitigation must rely on system design and user-interface improvements. Below are several possible interventions, ordered from most to least recommended:

1. **Introduce a recipient confirmation screen (“Are you sure?”)** — Adds mild friction, prompting users to re-check details. Not entirely effective, but significantly reduces accidental errors.

2. **Display the recipient's name for verification** — Provides strong feedback with minimal effort from the user. However, this raises privacy concerns, as malicious actors could infer personal information from a card number.
3. **Implement a QR-code based transfer option** — Allows recipients to generate a QR code that senders can scan. This reduces manual typing and is naturally preferred by users due to lower effort, making it a safer alternative.
4. **Adopt more robust validation algorithms beyond Luhn** — Technically feasible, but not recommended: existing cards would be incompatible, and rolling out new numbering systems is impractical.
5. **Adopt systems similar to the UK sort-code/account-number model** — Requires significant changes to national banking infrastructure and is therefore not recommended.

5 Conclusion

Monobank's transfer features show that user errors arise from manual 16-digit entry, lack of recipient feedback, and environmental or cognitive factors (SDM1, SM1–SM3, EDM3, EM1, EM3). Human limitations cannot be fully controlled, but interface interventions—confirmation screens, recipient name display, and QR-code transfers—reduce errors by lowering cognitive load and providing feedback. Infrastructure-heavy solutions are impractical. Targeting high-likelihood, high-impact risks demonstrates that secure, usable banking is achievable through thoughtful design, balancing usability and security in cyber systems.

References

- [1] R. Chatterjee, A. Athalye, D. Akhawe, A. Juels, and T. Ristenpart. Password typos and how to correct them securely, 2018. URL <https://www.cs.cornell.edu/~rahul/papers/pwtypos.pdf>.
- [2] D. Dawson and K. Reid. Fatigue, alcohol and performance impairment, 1997. URL <https://www.nature.com/articles/40775>.
- [3] R. Kainda, I. Flechais, and A. Roscoe. Security and usability: Analysis and evaluation, 2010. URL http://www.cs.ox.ac.uk/files/2859/ares_main.pdf.
- [4] Monobank. Official website, n.d. URL <https://monobank.ua/>.
- [5] Tviy Krok. Donations page, n.d. URL <https://tviykrok.com.ua/donations/>.
- [6] Wikipedia contributors. Luhn algorithm, n.d. URL https://en.wikipedia.org/wiki/Luhn_algorithm.

A Backend Code

```
1  """
2  Developed for HBCS coursework 1 by Artem Polovnev
3
4  Two-digit substitution Luhn-safety simulation
5
6  - Computes the Luhn check digit for each base to form a 16-digit test number.
7  - For each base, iterates all unordered pairs of positions starting from the 7th
  ↪ digit
8    (1-based positions 7..15), and for each pair tries every combination of two
  ↪ digits
9    (00..99). Skips the no-op case where both replacements equal the original
  ↪ digits.
10 - Records how many mutated bases produce the same Luhn check digit as the
  ↪ original.
11 - Outputs summary stats and some example matches.
12 """
13
14 import random
15 import string
16 from collections import defaultdict
17
18 def luhn_checksum_digit_from_base(base15: str) -> str:
19     assert len(base15) == 15 and base15.isdigit(), "base15 must be 15 digits"
20     digits = [int(c) for c in base15] + [0] # placeholder for check digit
21     total = 0
22     # Process from right to left
23     for i in range(len(digits)-1, -1, -1):
24         d = digits[i]
25         pos_from_right = len(digits) - i # 1-based
26         if pos_from_right % 2 == 0: # double every second digit from the right
27             d = d * 2
28             if d > 9:
29                 d -= 9
30         total += d
31     check = (10 - (total % 10)) % 10
32     return str(check)
33
34 def full_luhn_from_base(base15: str) -> str:
35     return base15 + luhn_checksum_digit_from_base(base15)
36
37 def simulate_two_digit_substitutions(card, verbose=False, keep_examples=20):
38     total_checks = 0
39     total_matches = 0
40     per_pair_matches = defaultdict(int)
41     example_matches = []
42
43     # positions: use zero-based indices; start from 7th digit => index 6
44     start_index = 6
45     end_index = 14 # inclusive (0-based index for 15th digit)
```

```

46 digits = '0123456789'
47
48 # generate all unordered pairs (i, j) with i < j, where i starts from
49 ↪ start_index
50 pairs = []
51 for i in range(start_index, end_index):
52     for j in range(i+1, end_index+1):
53         pairs.append((i, j))
54
55 base = card[:-1]
56 orig_check = luhn_checksum_digit_from_base(base)
57
58 for (i, j) in pairs:
59     orig_i = base[i]
60     orig_j = base[j]
61     # try all 100 combinations 00..99
62     for ri in digits:
63         for rj in digits:
64             # skip the no-op where both replacements equal originals
65             if ri == orig_i and rj == orig_j:
66                 continue
67             mutated = base[:i] + ri + base[i+1:j] + rj + base[j+1:]
68             total_checks += 1
69             mutated_check = luhn_checksum_digit_from_base(mutated)
70             if mutated_check == orig_check:
71                 total_matches += 1
72                 per_pair_matches[(i+1, j+1)] += 1 # store 1-based positions
73                 if len(example_matches) < keep_examples:
74                     example_matches.append({
75                         'base': base,
76                         'pos_pair': (i+1, j+1), # human 1-based
77                         'orig_digits': (orig_i, orig_j),
78                         'replacement': (ri, rj),
79                         'orig_check': orig_check,
80                         'mutated_base': mutated + str(orig_check)
81                     })
82
83 summary = {
84     'total_pairs_tested': len(pairs),
85     'total_mutation_checks': total_checks,
86     'total_matches_found': total_matches,
87     'match_rate': total_matches / total_checks if total_checks else 0.0,
88     'per_pair_matches': dict(per_pair_matches),
89     'examples': example_matches
90 }
91
92 if verbose:
93     print("Two-digit substitution simulation summary")
94     print("=====")
95     print(f"Total pairs considered: {len(pairs)}")
96     print(f"Total mutation checks performed:
97     ↪ {summary['total_mutation_checks'],}")

```

```

96     print(f"Total matches found (mutated base produced same Luhn check
    ↪ digit): {summary['total_matches_found']}")
97     print(f"Overall match rate: {summary['match_rate']:.6%}")
98     print("\nTop pairs by matches (sample):")
99     # sort pairs by matches descending and show top 10
100    sorted_pairs = sorted(per_pair_matches.items(), key=lambda kv: kv[1],
    ↪ reverse=True)
101    for (pair, cnt) in sorted_pairs[:10]:
102        print(f"    positions {pair}: {cnt} matches")
103    print("\nExample matches (up to {}):".format(len(example_matches)))
104    for ex in example_matches:
105        #print(ex)
106        print(ex["mutated_base"])
107        # Sorting out for the
108        #if ex["orig_digits"][0] == ex["replacement"][1] and
    ↪ ex["orig_digits"][1] == ex["replacement"][0] and
    ↪ (ex["pos_pair"][1] - ex["pos_pair"][0] == 1) :
109        #     print(ex)
110
111
112    return summary
113
114 if __name__ == '__main__':
115     # Example run (small number of bases so it runs quickly)
116     res = simulate_two_digit_substitutions(card = "4149499078909809",
    ↪ verbose=True, keep_examples=1000)

```